JDBC

Java DataBase Connectivity

- Direct SQL manipulation of relational databases
- Not an embedded SQL
- Queries represented as `String` values
- JDBC driver between Java and RDBMS
- DB-independence
- `java.sql` and `javax.sql`

JDBC provides abstraction over relational databases. It is considered a low-level API for persistence, e.g. compared to Java Persistence API (JPA) or to Java Data Objects (JDO) technologies, because:

1. it supports the direct SQL manipulation of the relational data, and
2. it fosters the construction and use of SQL commands and queries as `java.lang.String` values.

This latter is rather inconvenient and error-prone. A string value that is intended to represent a proper SQL term can be erroneously constructed, and in this case the error is detected at run-time, manifesting in an exception. Language embedding of SQL into Java would allow syntactic and semantic checking of SQL terms in compilation time.

However, JDBC is still a useful abstraction, since it allows (more or less) independence from concrete database management systems. Java code can access a relational database through the API provided by the `java.sql` and `javax.sql` packages, and the details are factored out into a program component called a JDBC-driver. For using a given RDBMS, the programmer is expected to obtain a JDBC-driver, which contains the RDBMS-specific implementation of the Java interfaces introduced by the `java.sql` and `javax.sql` packages.

Make it work

- Obtain JDBC driver
- How to load the driver?
  - Find out the name of the `Driver` class
  - Implements `java.sql.Driver`
- Find out the connection string syntax
  - `jdbc:sqlite:students.db`
  - `jdbc:mysql://localhost:3306/students`

Most of the RDBMSs are already equipped with one or more JDBC-drivers. Often they are freely available. They are typically shipped as a jar-file. Go and find the one you need on the web! During the course we will use Apache Derby, and its distribution called Java DB from Oracle.

Once you have the jar-file containing the JDBC driver around, you can start using databases managed by the corresponding database manager from your Java programs. In order to bind the driver to your program, you have to ``load'' the driver. There are different ways to do that, as we will see later on. Practically, a JDBC driver should contain a class that implements the `java.sql.Driver` interface. You need to know the name of this class to be able to load the driver. You can either find this name in the documentation of the driver, or you can simply unzip the jar file, and investigate the class files in it. The `javap` program in the Java SDK can help you find the right class.

There is one more thing to know: the syntax of the ``connection string''. This also should be described in the documentation of the driver. The connection string is used to specify which database to access (on which machine, on which port), together with additional information such as user name and password. The connection string typically looks like the ones above: it is a URI with a `jdbc`-scheme, followed by a subscheme identifying the RDBMS.

Write the code

- Loading the driver
- Create a `Connection`
- Use `java.sql` API
  - `Statement`
  - `ResultSet`
  - `PreparedStatement` etc.

The Java code that uses JDBC can make sure that the JDBC driver is loaded. Then it needs to create a connection using e.g. the `java.sql.DriverManager` class. The connection is represented as an instance of a (JDBC driver specific) class that implements the `java.sql.Connection` interface. Further important elements of the API are `Statement` and `ResultSet`.

Write-compile-run!

Simple example

```
import java.sql.*;
class SimpleDBExample {
  final String CONN_STR = "jdbc:derby:students.db";
  final String QUERY = "SELECT * FROM Grades";
```

```
  final String FIELD = "grade";
  public static void main(String args[]) throws SQLException {
     Connection c = DriverManager.getConnection(CONN_STR);
     try {
        Statement s = c.createStatement();
        try {
           ResultSet rs = s.executeQuery(QUERY);
           while( rs.next() ){
              System.out.println(rs.getInt(FIELD));
           }
        } finally { s.close(); }
     } finally { c.close(); }
  }
}
```

Here we can see a connection string for Derby, and a simple SQL query. JDBC throws the checked `java.sql.SQLException` whenever something goes wrong. We use the interfaces declared in `java.sql`, such as `Connection`, `Statement`, `ResultSet` etc. to type entities in the code. The dynamic types of the objects are provided by the JDBC driver. The workflow starts with the `DriverManager` class, and its static `getConnection` method. (It is overloaded to allow the specification of a user name and a password as well.) We get a `Connection`, and use it to create a `Statement`. One can execute multiple SQL commands with the same statement. The result of a query is represented as a `ResultSet`, which is idiomatically traversed by a `while` loop as above. If we query by primary key, the `ResultSet` will be of length zero or one. In this case the code uses a similar `if`-construct.

Each execution of the loop body will process one row in the result table. Within that row one can access the fields by name (or by position) with a getter of the appropriate type (`int` here).

Note how the code makes sure that the `Statement` and the `Connection` are closed after use (or at least their close operation are attempted), even if some exception is thrown.

Resources

- Connections and Statements
- Always close them!

It is extremely important to close `Statement` and `Connection` objects, since they may represent data outside of the JVM (e.g. an open result table inside the RDBMS). Even short-running Java SE applications should be careful enough with these resources, but in Java EE applications the proper closing of the resources is indispensable.

The code on the previous slide showed one way to structure the code in order to achieve an attempt to call `close` on both the statement and the connection. There

are other ways to structure the code. Java 7 makes this even easier with the *try-with-resources* construct.

Simple example in Java 7

```
import java.sql.*;
class SimpleDBExample {
  final String CONN_STR = "jdbc:derby:students.db";
  final String QUERY = "SELECT * FROM Grades";
  final String FIELD = "grade";
  public static void main(String args[]) throws SQLException {
     try( Connection c = DriverManager.getConnection(CONN_STR);
          Statement s = c.createStatement()
     ){
          ResultSet rs = s.executeQuery(QUERY) );
          while( rs.next() )
             System.out.println(rs.getString(FIELD));
     }
  }
}
```

This will only compile with Java 7 SDK. An attempt to close the resources in reverse order will be made. The try-with-resources construct is slightly different to the previous double try-finally structures in its strategy to suppress exceptions. Find out the details in the Java Language Specification!

How to compile and run?

- `javac SimpleDBExample.java`
- Is the DB manager running?
- `java -classpath .:derby.jar -Djdbc.drivers=org.apache.derby.jdbc.ClientDriver SimpleDBExample`
- `java` classpath jdbc-drivers program cmdline-args

The code in `SimpleDBExample` uses only standard API, so it can be compiled as it is. Running it requires some more efforts, however. We shall specify an appropriate classpath (containing the jar for the JDBC-driver), and a system property that will force the loading of the driver. UNIX syntax is used here, for other operating systems it need be slightly modified.

System properties are passed to the JVM with the `-D` flag. Here the system property `jdbc.drivers` is used. This property has a special meaning to the JVM: on startup it will attempt to load the class (or classes) specified in the `jdbc.drivers` system property. This is how the JDBC driver is ``loaded''. (Classes that implement the

`java.sql.Driver` interface are defined in such a way that during class initialization they register themselves into `java.sql.DriverManager`, using a static initialization block. This is how the `DriverManager` will know which drivers to ask for help when a connection request is made: it will ask all registered drivers until it finds one that supports the requested connection string.)

Note that the class(es) specified in `jdbc.drivers` must be possible to find in the class-path - most notably in (any of) the jar file(s) containing the JDBC driver(s).

Loading the driver

How to load the driver?

- Using jdbc.drivers
  - Flexible
  - Multiple drivers are supported
  - Fine for standalone applications
- Within the source code
- From JDBC 4: Service Provider mechanism

There are multiple ways to load the JDBC driver. The one we have seen is the use of the `jdbc.driver` system property. Since the code need not refer to the driver explicitly, it can be compiled before a decision on which RDBMS to use is made. So it's flexible. More than one drivers can be loaded by specifying a comma-separated list of class names in the system property, which is also fine. However, there is a disadvantage: this technique is mostly useful for applications started from the command line.

We shall investigate further mechanisms to load the driver.

Loading driver from code: static approach - 1

```
new org.apache.derby.jdbc.ClientDriver();
Connection c = DriverManager.getConnection(CONN_STR);
```

- Looks terrible
- Assumes a no-arg constructor

If an instance of the `Driver` class is created, the class is loaded into the JVM, and hence registers itself into `DriverManager`. It usually works, because these classes tend to have a no-arg constructor. However, it is ugly to construct an object for this side effect. Moreover, the code will statically depend on the driver - which might be undesirable in most cases.

Loading driver from code: static approach - 2

```
Class cl = org.apache.derby.jdbc.ClientDriver.class;
Connection c = DriverManager.getConnection(CONN_STR);
```

- Looks slightly better
- Hard-coded class name
- Statically checked class name

A slightly better approach is to explicitly refer to the appropriate class-literal. This will also load the class into the JVM, and result in the driver registering itself. However, the static dependence between the Java code and the driver still exists. This static dependence has an advantage, by the way. It forces the compiler to check the class name. This will prevent us from misspelling the class name. To sum it up, you will not find this solution too often in real-world code.

Loading driver from code: dynamic approach

```
Class.forName(NAME_OF_DRIVER_CLASS);
Connection c = DriverManager.getConnection(CONN_STR);
```

- Flexible
- Might throw checked exceptions

To eliminate static dependency, one can use `Class.forName`, which is the fundamental construct of reflection as well. You can read the name of the driver class from a configuration file (together with the connection string), or receive them as a parameter - it's quite flexible. Note, however, that different checked exceptions can be thrown by `forName`, and handling these can complicate the code.

You will see this solution quite often - too often to my taste...

Service Provider mechanism

- Service is an interface
  - `java.sql.Driver`
- Service Provider is an implementation
  - `org.apache.derby.jdbc.ClientDriver`

In the service provider jar file:

`META-INF/services/java.sql.Driver`:

`org.apache.derby.jdbc.ClientDriver`

Listing service providers

```
import java.sql.Driver;
import java.util.ServiceLoader;

public class ListDrivers {
  public static void main(String args[]) {
    ServiceLoader<Driver> sl = ServiceLoader.load(Driver.class);
    for(Driver d : sl) {
      System.out.println(d);
    }
  }
}
```

How to compile?

- Without driver jar
  - No explicit ref to driver
  - Special ways to start the program
- With driver jar in classpath
  - Explicit ref to driver

When the Java code statically depends on the JDBC driver, the jar file of the JDBC driver must be included in the classpath both for *compilation and execution*. When the Java code does not depend statically on the driver class name, the jar is only needed in the classpath for execution. In this latter case the Java code can be compiled without (prior to) selecting the RDBMS to use. This is often the preferable solution.

API

Connection pool

- Connections are expensive to open/close
- JDBC connections are not thread-safe
- Resource pooling
- `javax.sql.DataSource`

JDBC connections are designed for single-threaded use. If a server assigns a thread to each client, a separate connection should be used in each thread. Connections are typically quite heavy-weight resources, and the RDBMS does not allow too many simultaneous open connections. This limits the number of simultaneously served client

requests. If the application receives requests frequently, it is a good idea to close connections as early as possible. However, this leads to a huge number of connection open and close operations, which tend to be expensive (usually there are TCP-connections behind the curtains). Can we avoid the increased overhead? Connection pools are to our rescue. The standard way of obtaining connection pools is to use `javax.sql.DataSource` to acquire connections. This is a bit tricky to do in Java SE applications - you will learn about this during the labs. In Java EE applications you should practically *always* prefer `DataSource` to `DriverManager`.

Queries and updates

```
Statement s = ...
String QUERY  = "SELECT ...";
String UPDATE = "INSERT INTO ...";
String COLUMN_NAME = "...";
int affectedLines = s.executeUpdate(UPDATE);
ResultSet rs = s.executeQuery(QUERY);
while( rs.next() ){
  String str = rs.getString(COLUMN_NAME);
}
```

The simplest - but still quite frequent - use of JDBC has already been illustrated: we recap the workflow here. You can create several `Statement` objects from a `Connection`. Statements can be executed with ``query'' and ``update'' commands over and over again. A query returns a `ResultSet`, which is available until the `Statement` is used to execute another command, or is closed (or the `Connection` is closed). Although not seen too often in real world code, a `ResultSet` can also be closed. It is neither required nor important to close a `ResultSet` explicitly, because it will be closed automatically when the corresponding `Statement` is closed.

To improve efficiency, data from a `ResultSet` is often stored in the RDBMS, and fetched lazily - a field is passed through the network only when it is accessed.

Features of JDBC API

- Lots of additional stuff
- `ResultSet`: positioning, inserting, updating, deleting
- Java types and JDBC types
  - Java primitive types and `String`
  - `java.math.BigDecimal` - `NUMERIC`
  - Streams as `LONGVARCHAR` and `LONGVARBINARY`
  - `BLOB`, `CLOB`
  - `Date`, `Time`, `Timestamp`

The JDBC API has evolved over time - there were multiple specifications, each one introducing new features. JDBC offers lots of interesting features. Some of them will be addressed during the labs. The JDBC drivers for the most frequently used RDBMSs try to keep up with the changes of the specifications. However, it is possible that you cannot find a driver that complies with the latest standard for the RDBMS you intend to use. This will restrict the features available for your application. And, of course, the more sophisticated features you rely on in your application the less drivers you can use, and the less RDBMS you can deploy your application with.

Let's see a few examples on the features offered by `ResultSet`.

```
ResultSet rs = ...
rs.absolute(5);
rs.updateString("Name", "Janis Joplin");
rs.updateRow();
rs.moveToInsertRow();
rs.updateString(1,"John Lennon");
rs.updateInt(2,35);
rs.updateBoolean(3,true);
rs.insertRow();
rs.moveToCurrentRow();
rs.deleteRow();
```

Another important topic to study is the mapping between SQL types and the types offered by Java. Many SQL types (`BIT`/`BOOLEAN`, `TINYINT`, `SHORTINT`, `INTEGER`, `BIGINT`, `FLOAT`, `DOUBLE`) can be directly mapped to the primitive types of Java. `VARCHAR` is also naturally mapped to `String`. Other SQL types are mapped to Java SE API classes, e.g. `NUMERIC` to `java.math.BigDecimal`, and temporal types to corresponding classes in `java.sql`. The class `java.sql.Date` is a subclass of `java.util.Date`.

Hazard no. 1

Bad practice

```
import java.sql.*;
public class BadSql {
  public static void main(String[] args) throws SQLException {
    final String QUERY = "SELECT 1 FROM users WHERE " +
         "username='" + args[0] + "' AND password='" + args[1] + "'";
    try( Connection c = DriverManager.getConnection("jdbc:derby:sample");
         Statement stmt = c.createStatement()
    ){
         System.out.println(QUERY);  // just to see what's happening
         ResultSet rs = stmt.executeQuery(QUERY));
         if(rs.next())  // db contains the given username/password pair
```

```
            System.out.println("Username and password are correct.");
        }
    }
}
```

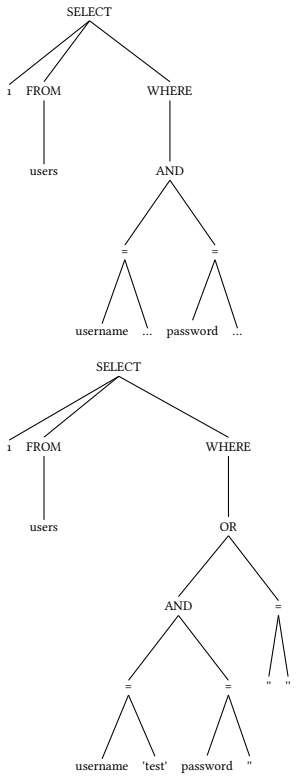
The code above illustrates wide-spread, and really very bad, practice. Since SQL commands are represented as `java.lang.String` values in Java, programmers often construct these command by concatenating strings dynamically. Apart from the fact that it is not esthetically pleasing, this is a source of many programming errors, and even of serious security holes.

This code checks whether a username/password pair is stored in the database. The two strings come from some user interface (for instance from a form in a web-based user interface, or, as in this very example, from the command line).

Can we hack this solution?

SQL-injection

```
$ java BadSql test "' OR ''='"
SELECT 1 FROM users WHERE username='test' AND password='' OR ''=''
Username and password are correct.
```



Definitely. Without knowing the password for the username ``test'' we can make the application think that we are authorized users. We have passed a tricky, cleverly constructed value as the second parameter. The application does not check whether the values passed as parameters are harmful - it just blindly concatenates them with the rest of the SQL command. The result is a valid SQL command - which means something very different to what the intentions of the programmer were.

The technique used here is referred to as SQL-injection. It works in situations when an SQL command is dynamically constructed from constituents received as parameters, without properly escaping, or at least thoroughly checking, the values of the parameters. The source of the problem is that the syntactic structure of the resulting command is not the same as it appears to be, or as the programmer intended to define. Due to the changes of the syntax, the expression tree describing the meaning of the command changes as well. In this example the programmer wanted the expression tree on the left, but the tricky second parameter resulted in the expression tree on the right.

PreparedStatement

```
import java.sql.*;
public class SqlPrepared {
  public static void main(String[] args) throws SQLException {
    final String QUERY = "SELECT 1 FROM users WHERE " +
                         "username=? AND password=?";
    try( Connection c = DriverManager.getConnection("jdbc:derby:sample");
         PreparedStatement stmt = c.prepareStatement(QUERY)
    ){
         stmt.setString(1, args[0]);
         stmt.setString(2, args[1]);

         ResultSet rs = stmt.executeQuery();
         if(rs.next())
           System.out.println("Username and password are correct.");
    }
  }
}
```

`java.sql.PreparedStatement` is a subtype of `Statement` that represents precompiled SQL commands: commands that contain holes for parameters can be sent to the SQL engine of the RDBMS, where they are compiled into some internal representation, and can be activated multiple times with different actual parameters. Not every RDBMS supports precompilation, however.

`PreparedStatement` offers two advantages.

1. Repeated execution of the same SQL command with different arguments becomes much more efficient.

2. SQL-injection can be prevented.

Short intro to transactions

Transactions

- Start, commit and rollback
- ACID properties
- `java.sql.Connection` API
  - Either automatic or explicit
  - So far: ``auto-commit''
    * Every statement executes in a new transaction
    * Upon completion of statement, the tx is committed
  - How to do it explicitly?

The regular way to handle concurrent accesses to a database is to use transactions. Transactions are units of work, constructed from query and data manipulation commands, often characterized by the ACID properties. Transactions are covered in another lecture in more details.

In a transaction API one needs, at least, to be able to start, commit and roll back transactions. How to do this in JDBC? There are two ways to use transactions in JDBC. The auto-commit approach means that the execution of each SQL statement is a separate transaction. This is the default behaviour in JDBC. One can switch to manual commit as well. All transaction-related operations are part of the `Connection` interface.

Explicit transaction in JDBC

```
import java.sql.*;
class TransferMoney {
  public static void main( String[] args ) throws SQLException {
    String fromId = args[0], toId = args[1];
    int amount = Integer.parseInt(args[2]);
    String UPDATE = "UPDATE Account SET balance=balance+? WHERE id=?";
    String CONN_STR = "jdbc:derby:accounts.db";
    try( Connection conn = DriverManager.getConnection(CONN_STR) ){
       conn.setAutoCommit(false);
       try( PreparedStatement stmt = conn.prepareStatement(UPDATE) ){
          stmt.setInt(1,-amount); stmt.setString(2,fromId);
          stmt.executeUpdate();
          stmt.setInt(1,amount); stmt.setString(2,toId);
          stmt.executeUpdate();
```

```
        }
        conn.commit();
    }
  }
}
```

The `setAutoCommit(false)` call will change the behaviour of the connection into manual transactions. The transaction now needs to be closed by calling either `commit` or `rollback` on the `Connection`. These calls will also start the next transaction. All SQL commands executed through `Statements` created from the `Connection` will belong to the current transaction.

Java Persistence API

What is JPA?

- Storing and retrieving relational data
- Connects Information System Tier and Business Logic Tier
- More convenient than JDBC
- More convenient than EJB2 Entity Bean

JPA is often used in Java SE and Java EE programs for manipulating relational data. It is higher level, and more convenient to use, than JDBC. Moreover, it replaces the Entity Bean technology found in earlier Java EE versions, and can be used easily to access EIS from business logic tier.

Overview

- JPA with Java SE possible
- Java EE web- and ejb-container
- Entities are Plain Old Java Objects (POJOs)
- Annotations or deployment descriptors (XML)
- Object-relational mapping standard
- Life cycle: EntityManager API
- DB operations: Query API

In this lesson we learn how to use JPA in Java SE applications, but later in the course our main goal with this technology will be to use it from JavaEE components. The most general approach is to access the database tier from EJB components, and here JPA provides a simpler, POJO-based replacement for the heavy-weight EJB 2 entity beans. One can easily specify entities using either annotations or XML descriptors.

JPA supports object-relational mapping, which makes it possible to describe a relational database in terms of classes and objects. The API introduces the concept of the Entity Manager, which handles life cycle of entities, and maintains the mapping between objects and tables. Bulk operations can also be performed with the Query API.

JPA at a glance

JavaBean POJO

```
public class Student {
    private int id;
    private String name;
    private byte grade;
    public Student() {}
    public Student(int id) { this.id = id; }
    public    int getId() { return id; }
    public   void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public   void setName(String name) { this.name = name; }
    public   byte getGrade() { return grade; }
    public   void setGrade (byte grade) { this.grade = grade; }
}
```

This is a very simple Java class definition for holding a record of three values. It is written in the JavaBeans style with public getters and setters providing access to private fields.

One can call objects of this class Plain Old Java Objects, POJOs. How can we make this represent a database table?

Entity

```
import javax.persistence.*;
@Entity public class Student {
    @Id private int id;
    private String name;
    private byte grade;
    public Student() {}
    public Student(int id) { this.id = id; }
    public    int getId() { return id; }
    public   void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public   void setName(String name) { this.name = name; }
```

```
    public   byte getGrade() { return grade; }
    public   void setGrade (byte grade) { this.grade = grade; }
}
```

By adding two annotations to the previous definition, we turn the POJO class into an Entity class. Objects of this class are called entities, and these can directly represent values in a database. We should mark the primary key of the table using the `Id` annotation.

There are some rules that must be respected, however. 1. The type declared as an Entity should not be an interface or an enum. It should not be a final class, either. It should be (an abstract or a concrete) top-level class. 2. The class must have a no-arg constructor. It can have further constructors as well. 3. The non-static non-transient fields and instance methods must not be final.

The entity class may extend other classes, even entity classes, and may be extended by (entity or non-entity) other classes. The entity class may be `Serializable` - this might come handy when we would like to pass values of this class to remote methods.

There is another strong requirement: instance fields of an object from the entity class may only be accessed by the methods of the same instance.

Entity Manager

```
EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("StudentService");


EntityManager em = emf.createEntityManager();

Student student = new Student(158);
student.setName(name);
student.setGrade(grade);
em.persist(student);


Student student = em.find(Student.class, 158);

student.setGrade(grade);
em.remove(student);
```

The entity manager can be used to manipulate the database through the state of the entity objects. The entity manager is instantiated by a factory class, which needs to be configured for a given database.

Instantiating an object from the `Student` class will create a Java object only. The object becomes an entity (we also say that the object becomes *managed*) when we perform

entity manager operations on it. The `persist` method will make the object managed, and ensures that a new row containing the current state of the object be inserted into the database. (To be more precise, the entity manager will insert the new row into the database at the end of the current transaction. All operations may run in transactions, and the data exchange between the Java program and the database takes place at the end of the corresponding transaction.)

Querying by primary key is done via the `find` method of the entity manager. The first parameter of this generic method is a type token: `find` can be called in a statically type-safe manner with this idiomatic solution. The returned object is managed again. Hence, setting the grade on this entity has an effect on the database - it will become an `UPDATE` statement in the relational world. Finally, the `remove` operation corresponds to a `DELETE`.

Queries

```
Query query = em.createQuery("SELECT s FROM Student s");
Collection students = query.getResultList();
```

```
em.createQuery("UPDATE Student s SET s.grade = s.grade + ?1 ")
  .setParameter(1, 1000)
  .executeUpdate();
```

Bulk operations, i.e. ones that might manipulate multiple lines in the database, can also be performed with the Query API. A special query language can be used for this purpose: it is similar to SQL, but resembles a bit to Java as well.

JPA without EJB

- JPA implementations available
  - TopLink (Oracle)
  - OpenJPA (Apache)
  - Hibernate
  - EclipseLink
- Easy to use JPA from JavaSE apps

One can download and use different JPA implementations in a Java SE application. The examples presented so far, for instance, can be configured easily for such a use.

JPA and EJB

```
@Stateless public class StudentBean implements StudentService {
  @PersistenceContext(unitName="StudentService")
  protected EntityManager em;

  public Student createStudent(int id, String name, byte grade) {
    Student student = new Student(id);
    student.setName(name); student.setGrade(grade);
    em.persist(student);
    return student;
  }

  public Collection<Student> findAllStudents() {
    Query query = em.createQuery("SELECT s FROM Student s");
    return (Collection<Student>) query.getResultList();
  }
}
```

When used in a Java EE container, configuration is to be performed by the administrator of the application server. The bean developer can inject a reference to an entity manager - we shall learn about beans and dependency injection later on.

Object-Relational Mapping

- Data Model
  - Values
  - Relations
- Customizing mapping
  - Tables
  - Columns

Object-relational impedance mismatch!

To define the ORM, one needs to specify the data model (what values to store, and what the relations are), and one may need to customize how classes map to tables, how fields map to columns and so on.

In many cases, or at least in simple ones, it is almost trivial to give a mapping between object networks and relational data. However, in more sophisticated applications, it turns out that certain difficulties, both conceptual and technical ones, may arise. This anomaly is referred to as *object-relational impedance mismatch*.

Specifying

- Annotation
- XML

The ORM can be given either with annotation or with XML descriptors. The former is more lightweight, and preferred by many programmers. The latter is more robust, and more expressive.

Data Model with Annotations

- Field based
- JavaBean based
- Fields:
  - `@Id`
  - `@Basic` (optional)
- Relations:
  - `@ManyToOne`
  - `@ManyToMany`
  - `@OneToMany`
  - `@OneToOne`

For simplicity, we are going to use the annotation-based approach. Note, however, that everything shown here is possible to describe in XML as well. XML descriptors can even describe details that annotations cannot capture.

Mapping data

Field based

```
import javax.persistence.*;

@Entity public class Student {
  @Id private int id;
  private String name;
  private byte grade;

  public    int getId() { return id; }
  public   void setId(int id) { this.id = id; }
  public String getName() { return name; }
  public   void setName(String name) { this.name = name; }
  public   byte getGrade() { return grade; }
  public   void setGrade (byte grade) { this.grade = grade; }
}
```

Revisiting our `Student` entity example, we can see that the `Id` annotation is placed on a field declaration of the class. There are two ways to add entity annotations. This is the field based way: all JPA-specific annotations on data columns and relationships are attached to field declarations.

(JavaBeans) property-based

```
import javax.persistence.*;

@Entity public class Student {
  private int id;
  private String name;
  private byte grade;

  @Id public int getId() { return id; }
  public    void setId(int id) { this.id = id; }
  public  String getName() { return name; }
  public    void setName(String name) { this.name = name; }
  public    byte getGrade() { return grade; }
  public    void setGrade (byte grade) { this.grade = grade; }
}
```

The other approach is to attach the annotations to JavaBeans getter methods. The two approaches are equivalent, but we have to decide which one we follow, and apply it consistently. I personally prefer the property-based approach, but this course material uses the field-based approach for the presentation (since it is shorter).

Declaring the table

```
@Entity
@Table(name="STUD")
public class Student { ... }

@Entity
@Table(name="STUD", schema="UNIV")
public class Student { ... }

@Entity
@Table(name="STUD", catalog="UNIV")
public class Student { ... }
```

One can customize some details of the object-relational mapping. Many details are sorted out with some defaulting mechanism, which can, however, be overridden.

Class names will be used as table names, for instance, by default. Using the `Table` annotation (or the equivalent XML tag) we can override this default behaviour, and specify the table name directly.

This customization approach is especially useful when we must adapt to existing databases.

Mapping Simple Types

- Primitive Java types: `byte`, `int`, `short`, `long`, `boolean`, `char`, `float`, `double`
- Wrapper classes of primitive Java types: `Byte`, `Integer`, `Short`, `Long`, `Boolean`, `Character`, `Float`, `Double`
- Byte and character array types: `byte[]`, `Byte[]`, `char[]`, `Character[]`
- Large numeric types: `java.math.BigInteger`, `java.math.BigDecimal`
- Strings: `java.lang.String`
- Java temporal types: `java.util.Date`, `java.util.Calendar`
- JDBC temporal types: `java.sql.Date`, `java.sql.Time`, `java.sql.Timestamp`
- Enumerated types: Any system or user-defined enumerated type
- Serializable objects: Any system or user-defined serializable type

So far we have seen that the state of the entity class defines the structure of the corresponding table, and the data fields will be directly mapped to columns with the respective type. What types can be used for such purposes? The list above shows the types that have an equivalent in the SQL world. These are the types that we can rely on in our entity classes.

Specifying Columns

```
@Entity public class Student {
    @Id @Column(name="STUD_ID") private int id;

    private String name;

    @Column(name="GRADE") private long grade;

    @Column(name="COMM") private String comments;

    // ...
}
```

Similarly to explicit `Table` declarations, we can specify the name of the table columns as well. The default column names are equal to the name of the fields (or the name of the JavaBeans properties, if property-based JPA annotations are used). This can be overridden with the `Column` annotation.

@Basic

```
@Entity public class Student {
    // ...

    @Basic(fetch=FetchType.LAZY)

    @Column(name="COMM")
    private String comments;

    // ...
}
```

The `Id` annotation is used for the primary key field. All other data fields can be annotated with the `Basic` annotation - which can also be omitted. Typically we use `Basic` only when we want to add some annotation elements to the annotation, such as `fetch`.

Lazy Fetching

```
@Entity public class Student {
    @Id private int id;

    @Basic(fetch=FetchType.LAZY) @Lob @Column(name="PIC")
    private byte[] picture;

    // ...
}
```

The `fetch` annotation element of the `Basic` annotation can be used to specify the fetching strategy for a particular field (or JavaBeans property). The default is eager fetching, but we can also switch to lazy fetching, as demonstrated by the example. Lazy fetching means that the entity manager will not load the value for that field (or relation, but this will be addressed later) when the entity is constructed from the database row. The value will only (if ever) be read from the database when the program code first tries to access it. The entity manager is responsible for performing this lazy fetching, which can improve performance by a great extent, especially if the field is large (a Large Object, `Lob`, in this case). Note, however, that there is a hazard associated to lazily fetched fields as well: if the entity becomes *detached* (e.g. we call the `detach` method of the entity manager with the entity) before we first access the respective field, no fetching will take place, and the access will find an empty field (zero or `null`).

Problem with Enums

```java
public enum Training {
    FULL_TIME,
    EVENING,
    CORRESPONDENCE
}

@Entity public class Student {
    @Id private int id;

    private Training type;

    // ...
}
```

- Mapping based on ORDINAL
- New value in the middle of enum?

Enumerated types are allowed to use in the object-relational mapping. Indeed, they seem quite easy to represent: since each enum value has an associated `ordinal` (in the example above, the three values have ordinal 0, 1 and 2), we can store this integer number as the respective column (e.g. a column `type` in the `Student` table).

This works. But software often changes. Such a change might introduce students in study-abroad training. Assume that for some reason, the programmer who introduces this new type of students, inserts a `STUDY_ABROAD` value to `Training` right after `FULL_TIME` and before `EVENING`. If the changed program still uses the database created with the original program, a strange bug will appear, one which is hard to track down: values in the database originally representing `CORRESPONDENCE` training will now represent `EVENING` training, since the change shifts their ordinals by one. So the above representation of enumerated values by their `ordinal()` values is sound - but fragile.

Alternative representation

```java
@Entity public class Student {
    @Id private int id;

    @Enumerated(EnumType.STRING)
    private Training type;

    // ...

}
```

There is another representation for enumerated types: we can store the enum values in the database as strings. One can choose either of the two representations with

the `Enumerated` annotation, which expects an `EnumType` annotation element, either `ORDINAL` or `STRING`. The default is `ORDINAL`.

The string representation tolerates changes in the number and order of enumeration values, but it consumes more space, and it breaks similarly when a code change affects the name of the enum values.

Temporal Types

```
@Entity public class Student {

    @Id private int id;

    @Temporal(TemporalType.DATE)
    private java.util.Calendar birth;

    @Temporal(TemporalType.DATE)
    @Column(name="REG_DATE")
    private java.util.Date registration;

    // ...
}

enum TemporalType { DATE, TIME, TIMESTAMP }
```

Another kind of data that deserves special attention is temporal data. Usually there are a number of different temporal types in the repertoire of a database engine. JPA supports three such types: `DATE`, `TIME` and `TIMESTAMP`. If a temporal value is used in the persistent state of an entity, we can choose using the `Temporal` annotation.

Moreover, there are different temporal types in the Java world as well. One can use `java.util.Calendar`, `java.util.Date`, and the `java.sql` temporal types too.

Transient Data

```
@Entity public class Student {
  @Id private int id;
  private String name;
  transient private String translatedName;

  public String toString() {
    if (translatedName == null) {
        translatedName = ResourceBundle
                          .getBundle("StudentResources")
```

```
                        .getString("student");
    }
    return translatedName + ": " id + " " + name;
  }
}
```

- `transient` or `@Transient`

Similarly to object serialization, we can declare object fields as *transient*, which means that those fields are not part of the persistent state, no database columns will be mapped onto them. You should use the `transient` keyword with field-based mapping, and the `Transient` annotation for JavaBeans-based mapping.

In transient fields one can store data that can be computed from other fields/data, hence it would cause unwanted redundancy in the database. In the example above, we use a localisation/i18n technique for acquiring the word meaning *student* in the language of the user of the program. Using a resource bundle, we look up the translation for *student* in the currently used language, and assign it lazily to `translatedName` in the `toString` method.

Entity inheritance

- Abstract entities
- Mapped superclasses
- Non-Entity Superclasses

Entities support class inheritance, polymorphic associations, and polymorphic queries. Entity classes can extend non-entity classes, and non-entity classes can extend entity classes. Entity classes can be both abstract and concrete.

Both abstract entities and mapped superclasses may be a superclass of an entity with the following differences.

- Abstract entities can be queried like concrete entities. Querying abstract entities will yield all entities which inherit from the abstract entity independently of their concrete type. How entities are mapped to a database can be specified explicitly.
- Mapped superclasses cannot be queried - a concrete entity class must be used. In this case concrete entities are typically mapped to separate tables in database.

An entity class may inherit from a non-entity class, but the state of non-entity superclasses is non-persistent, and any state inherited from the non-entity superclass by an entity class is non-persistent.

Inheritance Mapping Strategies

`javax.persistence.Inheritance`

- SINGLE_TABLE
- TABLE_PER_CLASS
- JOINED

The way JPA maps inherited entities to the underlying database can be configured by decorating the root class of the hierarchy with the annotation `javax.persistence.Inheritance`. There are three mapping strategies.

1. SINGLE_TABLE tells JPA that a single table containing all persistent fields of all derived entities should be created. All fields of the derived classes are created nullable in the database. Only fields of the actually represented entity are filled with values. In this case the concrete type of the entity must be stored for every row in the database in order to make mapping unambiguous. Because of this an entity must contain an either generated or explicitly defined (annotations or xml descriptor file) *discriminator column*.
2. TABLE_PER_CLASS tells JPA that a separate table should be created for each derived entity class. In this case representation is unambiguous, but querying entities of base classes will result in the UNION of the tables assigned to the derived entity classes.
3. JOINED - In this case the root of the class hierarchy is represented by a single table, and each subclass has a separate table that contains only those fields specific to that subclass. That is, the subclass table does not contain columns for inherited fields or properties. The subclass table also has a column or columns that represent its primary key, which is a foreign key to the primary key of the superclass table. This strategy provides good support for polymorphic relationships but requires one or more join operations to be performed when instantiating entity subclasses, which may result in poor performance for extensive class hierarchies. Similarly, querying entities of base classes results in JOIN and UNION operations at SQL level. Additionally, in this case, some JPA implementations also require the presence of a discriminator column, as described above.

Primary keys

Identifier

- Primitive Java types: byte, int, short, long, char
- Wrapper classes of primitive Java types: Byte, Integer, Short, Long, Character
- Arrays of primitive or wrapper types: Byte, Integer, Short, Long, Character

- Strings: java.lang.String
- Large numeric types: java.math.BigInteger
- Temporal types: java.util.Date, java.sql.Date

Never use floating type

One of the key issues in relational databases is the use of *identifiers*. Much of the object-relational mapping can be defined implicitly, using the default mechanisms, but the identifier of the entities must be explicitly specified, at a minimum. The `Id` annotation is used for this purpose, as we have seen in the previous examples.

An identifier can be represented as a primary key in the database. There are certain rules what kinds of data are safe to use as identifiers. The rule of thumb is that we should never use floating point types. Equality checking is the most important operation on identifiers, and this operation behaves badly for floating point types.

Generated Identifiers

- Automatic Id Generation
- Id Generation Using a Table
- Id Generation Using a Database Sequence
- Id Generation Using Database Identity

When we create an entity of a given type, its identifier must be assigned a valid and unique value. It is not always possible to point out a field in an entity class such that it is unique and its value is always known for each entity. Sometimes the programmer cannot find an appropriate identifier, and/or just decides to use an artificial identifier instead. These identifiers can convenient to use, because they can be automatically generated. There are four ways to implement this with the JPA object-relational mapping.

Automatic Id Generation

```
@Entity public class Student {
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int id;

    // ...
}
```

- Mostly for development time

With automatic ID generation, we use the `GeneratedValue` annotation, and specify `AUTO` as the strategy. In this case JPA will decide which method to use from the forthcoming three approaches.

This is a convenient, but fragile approach, since in this case the programmer allows the system to make a decision, which might have an impact on the non-functional, or in some cases even in the functional, properties of the code. Use this strategy with care! The best is to use it for development time, and replace it with an explicit decision for the production code.

Id Generation Using a Table

```
@Id @GeneratedValue(strategy=GenerationType.TABLE)
private int id;

@TableGenerator(name="Student_Gen")
@Id @GeneratedValue(generator="Student_Gen")
private int id;

@TableGenerator(name="Student_Gen",
    table="ID_GEN",
    pkColumnName="GEN_NAME",
    valueColumnName="GEN_VAL")
```

The `TABLE` value for `strategy` specifies that a special table is to be used to store auxiliary data for ID generation. Again, default mechanisms apply, but the programmer can make explicit decisions on the details of the ORM as well. The `TableGenerator` annotation can provide information to the ORM at different levels of details.

In the bottom-most, fairly detailed, annotation we specify that the `ID_GEN` table should be used by the DB to store auxiliary data for ID generation. The table should have a column, `GEN_NAME`, for storing the name of the entities that need automatic ID generation, and it should also have a column `GEN_VAL` that stores the *next ID* for those entities.

Id Generation Using a Table

```
@TableGenerator(
    name="Professor_Gen", table="ID_GEN",
    pkColumnName="GEN_NAME", valueColumnName="GEN_VAL",
    pkColumnValue="Prof_Gen", initialValue=1024,
    allocationSize=64)
```

ID_GEN

GEN_NAME	GEN_VAL
Student_Gen	0
Prof_Gen	1024

Here we can see the `ID_GEN` table, and that the next assignable ID for the Student entity is 0. We can specify even more details, like we do for professors here.

Id Generation Using a Database Sequence

```
@Id @GeneratedValue(strategy=GenerationType.SEQUENCE)
private int id;

@SequenceGenerator(name="Student_Gen", sequenceName="Student_Seq")
@Id @GeneratedValue(generator="Student_Gen")
private int id;

CREATE SEQUENCE Student_Seq
MINVALUE 1
START WITH 1
INCREMENT BY 8
```

Certain DB engines support database sequences. The technique presented here is suitable for these back-ends. The `GeneratedValue` annotation tells us that `id` draws its values from the `Student_Gen` generator. The `SequenceGenerator` annotation tells us that this generator is implemented by the `Student_Seq` sequence, which can be defined as shown above.

Id Generation Using Database Identity

```
@Id @GeneratedValue(strategy=GenerationType.IDENTITY)
private int id;
```

- Autonumber column
- No value before inserting

Finally, if the database engine supports auto-numbered rows, we can rely on this feature for ID generation. In certain databases a column in a table can be declared *auto number* or *identity*. When we add a new row into that table, the value of that column will be set automatically to the larger-by-one value with respect to the previous row. Note, however, that entities of this kind will not have a valid ID in ther identifier field until the entity is actually inserted into the database, typically at the end of the current transaction.

Composite identifier

- ID made of more fields
- Corresponds to composite primary key
- Two ways to achieve
  - `IdClass`
  - `EmbeddedId` and `Embeddable`

Sometimes the natural identifier of an entity is a composite value. This is the case when one uses a composite primary key in a relational database. There are two ways to represent composite identifiers in JPA. Now we will consider how to describe these two approaches with annotations.

IdClass

Entity class

```
@Entity @IdClass(buildings.RoomID.class) public class Room {
    @Id String building;
    @Id short number;
    short capacity;
    ...
}
```

ID class

```
package buildings;
public class RoomID implements java.io.Serializable {
    public String building;
    public short number;
    @Override public boolean equals( Object other ){...}
    ...
}
```

The `Room` entity has two fields, `building` and `number`, that together make up the identifier for this entity. Both of them are declared as `@Id`. The very same fields reappear in the class given by the `IdClass` annotation, here `RoomId`. The primary key class must respect certain rules.

- It should be a public class with a public no-arg constructor.
- It should also be `Serializable`.
- It must override `equals` (and `hashCode`, consequently) in such a way that it is consistent with the equality check executed by the database.

EmbeddedId and Embeddable

Entity class

```
@Entity public class Room {
    @EmbeddedId RoomID id;
    short capacity;
    ...
}
```

ID class

```
@Embeddable public class RoomID implements Serializable {
    private String building;
    private short number;
    @Override public boolean equals( Object other ){...}
    ...
}
```

The fields in the embedded class will become columns in the entity table, just like the persistent fields of the entity class. These columns form the primary key of the table.

Mapping relationships

Relationships

- Navigability
- Direction
- Multiplicity
- Optionality

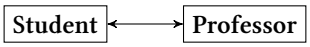
The mapping of object relationships, that is associations among entities, to relational databases is even more intricate than the mapping of object fields to columns. Object relationships are realized as objects storing references to other objects. In relational databases we can use foreign keys and join tables to implement this. According to the JPA approach to ORM, we need to investigate object relationships from the point of view of the entities.

We shall understand four important properties of relationships. If we have two associated objects, *navigability* tells us whether we can navigate between the two in both directions, or only from one. If we can only navigate from one object to the other, but not the other way round, *direction* tells us which is the source, and which

is the target of navigation. *Multiplicity* describes whether an object is in the given relationship with only one other object, or with multiple objects. In the latter case we need to store a collection of references. Finally, *optionality* of the relationship tells us whether it is allowed for an object not to be associated to other objects. If an entity stores a reference to another object, and the corresponding association is optional, the reference may be `null`. However, if the association is not optional, the reference must always point to a valid object. In SQL, the `NOT NULL` constraint represents the concept of (non-)optionality.

Navigability

Bidirectional relationship



Unidirectional relationship

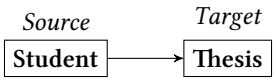


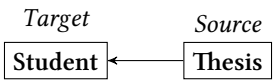
According to the navigability requirements of the program logic, one can introduce bidirectional and unidirectional relationships between entity classes. If we would like to store, and be able to retrieve the supervisor of a student, as well as the supervised student(s) of a professor, we use a bidirectional relationship. Both entity classes will have the corresponding references (fields), referring to objects of the other entity class.

If we want to be able to retrieve the students who enrolled for a given course, but we need not be able to retrieve the courses a students enrolled for, we can define a unidirectional relationship between the two entity classes. There will be a field in the `Course` class that refers to `Student` entities, but there will be no field in the `Student` class referring to `Course` entities.

Direction

Unidirectional relationships





When a student and a thesis are associated to each other, the application logic requires us to decide whether the relationship should be navigable from both directions, or

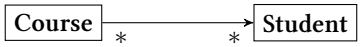
only from one direction. If the relationship is unidirectional, the application logic will also imply the *direction* of the navigation. The application might require us to be able to find the thesis of a given student - this is the relationship on the top. The application might require us to be able to find the preparing student for any given thesis - this relationship is demonstrated on the bottom.

Multiplicity

Bidirectional many-to-one



Unidirectional many-to-many



Multiplicity of a relationship tells us whether one or more entities are associated to one or more entities. We could have a bidirectional relationship between students and supervising professors. Many students can be supervised by the same professor, but a student is supervised only by one professor.

We could also have a unidirectional relationship between courses and students, where we can tell which students attend a given course. (Since this relationship is unidirectional now, we cannot easily tell which courses a given student attends.) This relationship is many-to-many, which means that a course can have many students, and a student can have many courses.

Multiplicity options

1. Many-to-one
2. One-to-one
3. One-to-many
4. Many-to-many

These are the four options for multiplicity.

ManyToOne

```
@Entity public class Student {
    // ...
```

```
    @ManyToOne private Institution home;

    // ...
}
```

- single valued

Here we can see a many-to-one relationship: each student is assigned to a (single) home institution, and many students may be assigned to the same home institution. This is called a *single valued* relationship, because it is a *to-one* relationship. Single valued relationships are represented with a reference of the other entity type.

The database mapping will include a column in the source table that contains a foreign key to the target (the other entity) table.

ManyToOne + JoinColumn

```
@Entity public class Student {
    // ...

    @ManyToOne @JoinColumn(name="INST_ID")
    private Institution home;

    // ...
}
```

One can customize the object-relational mapping of the relationship. For example, we can specify the name of the foreign key column.

OneToOne

```
@Entity public class Faculty {
    @Id private int id;
    private String name;

    @OneToOne private Professor dean;

    // ...
}
```

In this relationship we can tell who is the dean of a given faculty. Each faculty has exactly one dean, and a professor can be a dean at single faculty. Hence this is a one-to-one mapping. This could be a unidirectional relationship, and its direction is from the faculty to the professor.

Bidirectional OneToOne

Owning side

```
@Entity public class Faculty {
    @Id private int id;
    private String name;
    @OneToOne private Professor dean;
}
```

Inverse side

```
@Entity public class Professor {
    @Id private int id;
    @OneToOne(mappedBy="dean") private Faculty faculty;
}
```

We can turn the relationship into bidirectional by adding a matching field into the target class. Now both entity classes have a reference to the other, both are `OneToOne`, but the situation is not symmetric. We need to break the symmetry in order to decide which table contains the foreign key. This table will be called the *owning side* of the relation, and the other will be the *inverse side*. Note how the two sides are associated with the `mappedBy` annotation element!

Since not every professor is a dean, the relationship is optional from the point of view of professors. A faculty must always have a dean, so the relationship is not optional from the point of view of faculties.

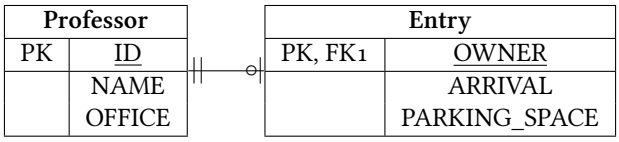
Identifying relationship

```
@Entity public class Professor {
  @Id private int id;
  private String name;

  @OneToOne(mappedBy="owner")
  private Entry carParkEntry;
  //...
}
```

```
package underground_car_park;
@Entity public class Entry {
  private Time arrival;
```

```
  @Id @JoinColumn
  @OneToOne(optional=false)
  private Professor owner;
  //...
}
```



In a one-to-one relationship it makes sense to share identifiers. Let's see an example. In the underground car park only professors are allowed to enter. We want to record entries into this car park. Each entry entity will be identified by the owner of the car. Each professor can enter into the car park with at most one car at a time (albeit a professor may have more than one car, each allowed to enter the car park). So this is a one-to-one relationship, which is optional at the professors' side (a professor may have not entered the underground car park), but not optional from the point of view of entries. The primary key for entries will be the same as the foreign key referring to the car owner (professor), so the identifier of the `Entry` entity will be actually the professor.

When a relationship (a foreign key) is part of the identifier (the primary key), we say that it is an identifying relationship.

OneToMany

```
@Entity public class Professor {
    @Id private int id;

    @OneToMany private Set<Student> researchStudents;

    // ...
}
```

- unidirectional, optional, one-to-many
- collection-valued relationship

This relationship describes that a professor may work together with some students on research topics. Many professors, but only a few students are engaged in such an activity, and the application logic will probably require us to navigate only from professors to students. This will lead us to a unidirectional, optional, one-to-many relationship.

Because of *to-many*, we call this relationship *collection-valued*. The referring entity class will have a reference to a collection of the referred entity class. We can only use the Collections Framework interfaces (`Collection`, `Set`, `List`, `Map` from `java.util`) to type these references.

Without parametrized types

```
@Entity public class Professor {
    @Id private int id;

    @OneToMany(targetEntity=Student.class)
    private Set researchStudents;

    // ...
}
```

- Old style, raw type
- Most probably with XML

It is possible to achieve the same effect without parametrized types. This may be important when relying on some old code base, one from the pre-generic era. In this case we have to specify the target entity of the relationship as part of the ORM. The example above uses annotations for this, but XML-based mapping seems even more realistic when we work with some old code base, and we have no possibility or right to modify the Java source.

Bidirectional ManyToOne/OneToMany

Owning side must be the many-to-one

```
@Entity public class Student {
    @Id private int id;

    @ManyToOne private Professor supervisor;

}
```

Inverse side must be the one-to-many

```
@Entity public class Professor {
    @Id private int id;
```

```
    @OneToMany(mappedBy="supervisor")
    private Set<Student> supervisedStudents;

}
```

We have seen already a one-to-one bidirectional relationship. Now let's investigate an asymmetric case, where the different sides have different multiplicities: many-to-one and one-to-many. The many-to-one side is a single-valued relationship, and the one-to-many side is a collection-valued one.

There is a strict rule for this case: the many-to-one side must be the owning side, and the one-to-many side must be the inverse side, where the `mappedBy` is used. This rule is accordance with the way relationships are modelled in the ORM with foreign keys. This relationship can be implemented easily by defining a foreign key column in the *many* side, because from that side the relationship is single valued. This is the most natural way to implement a many-to-one/one-to-many bidirectional relationship.

Forgetting mappedBy

- It will compile, it will run, it will mean something else…
- Bidirectional relationship: synchronized data

Two independent unidirectional relationships

```
@Entity public class Student {
    @Id private int id;
    @ManyToOne private Professor supervisor;
}
//-------------------------------------------------------
@Entity public class Professor {
    @Id private int id;
    @OneToMany private Set<Student> supervisedStudents;
}
```

A potential source of error is omitting the `mappedBy` from the inverse side. Unfortunately that is also meaningful: it will compile, it will work, but it will behave very differently. The bidirectional relationship means that consistency is preserved for the two sides. If the supervisor of a student is changed, the supervised students of the respective professors change as well. Without `mappedBy` the bidirectional relationship becomes two completely independent unidirectional relationships.

Bidirectional ManyToMany

```
@Entity public class Student {
    @Id private int id;
    private String name;

    @ManyToMany private Collection<Course> currentCourses;
}
//-------------------------------------------------------
@Entity public class Course {
    @Id private int code;

    @ManyToMany(mappedBy="currentCourses")
    private Collection<Student> enrolledStudents;
}
```

The bidirectional many-to-many relationship is symmetric. One side should be arbitrarily chosen as inverse side with `mappedBy` to ensure proper bidirectional behaviour.

Since both sides are collection-valued, we should choose one of the `java.util` collection interfaces to provide the static type of the references. (It is always a good idea to use parametrized types.) The dynamic type of the references will be JPA-specific, so never use types other than the interfaces defined in the standard Collection Framework! The interface `java.util.Collection` is not always the best choice, however...

Using Set and List

```
@Entity public class Student {
    @Id private int id;
    private String name;

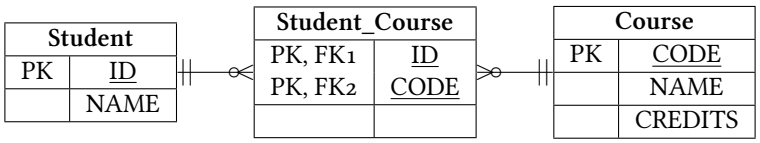
    @ManyToMany private Set<Course> currentCourses;
}
//-------------------------------------------------------------
@Entity public class Course {
    @Id private int code;

    @ManyToMany(mappedBy="currentCourses") @OrderBy("name ASC")
    private List<Student> enrolledStudents;
}
```

The static type `Set` emphasizes unicity, i.e. no duplicates can be present in the collection. The static type `List`, on the other hand, can express an ordering in the collection. This can be accompanied with `OrderBy`: in our example we can traverse the students enrolled for a course in ascending order of their names.

Collection-valued relationship: join table

- Bidirectional many-to-many
- Unidirectional many-to-many
- Unidirectional one-to-many



Collection-valued relationships are implemented with join tables. The only exception is a bidirectional many-to-one/one-to-many which can also be implemented with a foreign key in the *many* side.

@JoinTable

```
@Entity public class Student {
  @Id private int id;
  private String name;

  @JoinTable(name="STU_COUR",
             joinColumns=@JoinColumn(name="STU_ID"),
             inverseJoinColumns=@JoinColumn(name="COUR_ID"))
  @ManyToMany private Set<Course> currentCourses;
}
```

We can customize the ORM with the `JoinTable` annotation, specifying the name for the join table and for its columns.

Map

```
@Entity public class Student {
    @Id private int id;
    private String name;

    @ManyToMany private Set<Course> currentCourses;
}
//-----------------------------------------------------------
@Entity public class Course {
    @Id private int code;

    @ManyToMany(mappedBy="currentCourses") @MapKey(name="name")
    private Map<String, Student> enrolledStudents;
}
```

With the `java.util.Map` interface we can boost up a collection-valued relationship so that we can access related entities by a key. If we intend to traverse the students enrolled for a course by ascending order of their names, the previous entity definition is just fine. But if we intend to look up enrolled students by name, we need to perform a binary search. To eliminate the need for binary search, we can use a `Map` with `name` as the key.

Lazy Fetching

```
@Entity public class Student {
    @Id private int id;

    @ManyToMany(fetch=FetchType.LAZY)
    private Set<Course> currentCourses;

}
```

- More typical for relationships than for values
- Effect on performance
- Take care with detached entities

We have already seen how lazy fetching can be used to improve performance by postponing the loading of large data items until they are actually needed by the application logic. This lazy-fetching technique is even more important for relationships. Eager loading of a relationship may be very costly, especially if applied transitively.

Of course, the problem with detaching still holds: if an entity becomes detached before fetching associated entities, we can bump into `null` references or empty collections very easily.

Entity Manager

Life-cycle of entity instances

- New (no associated persistence identity)
- Managed
- Detached
- Removed

Basic functionality

- persist

- merge
- remove
- find
- refresh
- flush
- attach/detach

The `EntityManager` API creates and removes persistent entity instances, finds entities by primary key of the entity, and allows queries to be run on entities. Instances of this interface are typically provided by the framework running the code such as an application server. These instances are created, managed and propagated by the container, this way hiding the complexity of resource sharing (such as persistence context and transaction management).

Application-managed `EntityManager` instances can be created by using the `createEntityManager` method on the instances of `EntityManagerFactory`, which can also be acquired from the container (if present), or created using the `createEntityManagerFactory` static method of the `Persistence` class. The latter case only make sense when not running within a container, and should not be used in Java Enterprise applications. We will discuss the container related functionality in later lessons.

Entities can be managed by invoking operations on the instance of the `EntityManager` interface. Entities have the 4 states listed above. Once a new instance of an entity class is passed to `persist` method (or is associated to an instance of an entity passed to `persist` and the relationship cascading contains the `PERSIST` flag), it is moved to the *New* state. Once the transaction associated with the `persist` operation is completed (i.e. data is stored to the database), the entity becomes *Managed*. If the `persist` operation happens on a *Managed* entity either explicitly or through the operation propagation (cascading), the database is only updated when the entity is ``dirty'' (i.e. contains unsaved data).

A *Managed* entity can become detached when using the `detach` method. Passing a *Detached* entity to the `persist` operation will either result in `IllegalArgumentException` or transaction rollback. *Detached* entities can once again be attached to the persistence context with the `attach` operation. Detaching and reattaching entities typically performed when the entity is transfered to another application or another persistence context (e.g. when passed on through a remote interface).

A *Managed* entity can also be removed. It is then moved into the *Removed* state, and all corresponding data will be removed from database once the transaction associated with the `remove` operation is finished. If a *Removed* entity is persisted, it becomes *Managed* again.

The `find` method allows us to fetch an entity instance with the specified primary key. Entities returned by this method are automatically in *Managed* state.

The state of a *Managed* entity can be updated, i.e. reloaded from the database using the `refresh` method. Finally, synchronization of the current state can be forced using the `flush` method.

Cascading

`javax.persistence.CascadeType`

- `ALL`
- `DETACH`
- `MERGE`
- `PERSIST`
- `REFRESH`
- `REMOVE`

Entities with relationships often have dependencies on the existence of the other entity in the relationship. For example, a line item is part of an order; if the order is deleted, the line item should also be deleted. This is called a *cascade delete* operation.

The `javax.persistence.CascadeType` enumerated type defines the cascade operations that can be applied in the `cascade` element of the relationship annotations on the owning side. The following list explains the cascading possibilities. Note that one can specify more than one cascade operation for a given relationship.

- `ALL`: All cascade operations on the parent entity will be applied to the related entity as well. `ALL` is equivalent to specifying `cascade={DETACH, MERGE, PERSIST, REFRESH, REMOVE}`.
- `DETACH`: If the parent entity is detached from the persistence context, the related entity will also be detached.
- `MERGE`: If the parent entity is merged into the persistence context, the related entity will also be merged.
- `PERSIST`: If the parent entity is persisted into the persistence context, the related entity will also be persisted.
- `REFRESH`: If the parent entity is refreshed in the current persistence context, the related entity will also be refreshed.
- `REMOVE`: If the parent entity is removed from the current persistence context, the related entity will also be removed.

Queries

The Java Persistence Query Language

- Queries

  - dynamic
  - static (named)
- Parameters
  - named
  - positional

The Java Persistence Query language (JPQL) defines queries for entities and their persistent state. The query language allows creation of portable queries that work regardless of the underlying data store.

JPQL queries are parsed at runtime, and translated using the available metadata of the entities. Then a database specific query is created and executed. JPQL queries can be created dynamically using the `createQuery` method (dynamic queries), or defined in metadata using the `javax.persistence.NamedQuery` annotation, and accessed at runtime using the `createNamedQuery` method (static or named queries). In both cases a JPQL query may contain named parameters (prefixed with colon in queries) and positional parameters (prefixed with question mark in queries). These parameters are to be set at runtime prior getting results using the `setParameter` method. Although JPQL allows both named and positional parameters, they cannot be mixed in a single query. Results can be fetched using either the `getSingleResult` or the `getResultList` methods.

JPQL operations

- Select
  - FROM clause
  - Path expressions
  - WHERE clause
  - SELECT clause
  - ORDER BY clause
  - GROUP BY/HAVING clause
  - DISTINCT
- Update
- Delete

JPQL allows the creation of 3 types of queries. None of the queries are type safe. Selections (or select queries) provide a mechanism for grouping, ordering, filtering and querying generic data from the database. Update queries allow the updating of many entities at once. Deletions (or Delete queries) allow the removing of multiple items from the database. All of them may have a WHERE clause, which is also used for filtering results in selections.

The Criteria API

- Closely related to JPQL
- Object-level operations
- Metamodel API
- Typesafe queries

The Criteria API is used to define queries for entities and their persistent state by creating query-defining objects. Criteria queries are written using Java programming language APIs, are typesafe, and are portable. Similarly to JPQL, queries created using Criteria API work regardless of the underlying data store.

The Metamodel API

- Metamodel of the managed entities
- Same package
- Trailing underscope
- Attributes correspond to persistent fields/properties
- Used in Criteria queries

The Metamodel API is used to create a metamodel of the managed entities in a particular persistence unit. For each entity class in a particular package, a metamodel class is created with a trailing underscore, and with attributes that correspond to the persistent fields or properties of the entity class.

Metamodel classes can be generated by annotation processors either at development time or at runtime. After the generation they can be obtained by using the `getModel` method of the `Root` object returned by the `CriteriaQuery.from` method, or by using the `entity` method of the metamodel object returned by the `EntityManager.getMetamodel` method.

Criteria API Queries

- Query roots
- Relationships (Joins)
- Path navigation
- Restrictions (Selectors)
- Ordering
- Grouping

The Criteria API exposes the same functionality as JPQL, but the creation of the queries is performed using Java programming language APIs instead of parsing

queries from strings. This implies that either the query creation will not compile or the query will be well-formed and type-safe. Building queries is performed through instance of the `CriterialBuilder` object which can be obtained by using the `EntityManager.getCriteriaBuilder` method.